

Software Architecture

EE 8217/EC 8208



Based on Software Architecture in Practice (3rd Edition)
By Len Bass, Paul Clements, Rick Kazman

Outline

Why Is Software Architecture Important?

- Traditional Software Development Problem
- Emergence of Software Architecture
- Importance of Architectural Structures
- Architectural Quality Attributes

Traditional Software Development Problem

- Previously, software development followed this idea:

Requirements → Design → System

- Developers believed:

If technical requirements are correct, a good system will automatically result.

Can a system be functionally correct but still fail in real life?



Have you modified old code and felt scared to touch it?

Problem with This Approach

- This approach ignored:

Scalability ,Performance ,Security ,Maintainability
,Future changes

- Systems became:
 - Hard to modify
 - Difficult to maintain
 - Complex over time
 - Fixing becomes expensive.

Emergence of Software Architecture

- Developers realized:

Technical requirements alone are not enough.

- Architecture helps design systems that satisfy:
 - Quality attributes
 - Business goals
 - Long-term evolution

**A system may work correctly today,
but without good architecture, it may
fail tomorrow when requirements,
scale, or technology change.**



Why Software Architecture is Important?



1. Enables or Inhibits Quality Attributes

**What are these
Quality Attributes?**



2. Helps Manage Change

- Good architecture makes future modifications easier.

Example:

Adding a payment gateway becomes easier in modular systems.

3. Enables Early Analysis

A good architect can predict:

- Performance
- Risks
- Scalability issues

before implementation.

4. Improves Communication

- Architecture diagrams help:
 - Developers
 - Managers
 - Clients
- understand the system.

Architecture becomes a communication language.

5. Contains Fundamental Design Decisions

Architecture includes important decisions such as:

- Technology stack
- Deployment style
- Database structure

These are difficult to change later.

6. Defines Constraints

- Architecture sets rules for development.

- **Example:**

In layered architecture, UI cannot directly access database

7. Influences Organizational Structure

- Teams are often organized according to architecture.

- **Example:**

Separate teams for:

- Frontend
- Backend
- Database

10. Supports Product Lines & Reusability

- Reusable architectures help create multiple related products.
- **Example:**

Android phone manufacturers reuse common architectures.

- A good software architecture is necessary, but by itself it is not enough to guarantee software quality.
- Quality attributes depend on:
 - Architecture
 - Implementation
 - Deployment



Architecture Provides the “Big Picture”

- Focuses on:
 - Overall system structure
 - Component interactions
 - System organization
- It defines the high-level design of the system.

Poor Coding Can Ruin Good Architecture

- Even with excellent architecture:

Obscure or tangled code can make the system difficult to modify and maintain.

- **Example:**
 - Bad variable naming
 - Repeated code
 - Complex logic
 - No documentation

Software architecture is important because it determines whether a system can survive growth, change, complexity, and real-world usage.



Stakeholders in Software Architecture

Who is a stakeholder?

There's always one more System Stakeholder than you know about; and the known stakeholders have at least one more need than you know about now.

-Tom Gilb-

- A stakeholder is a person or organization that has rights, shares, claims, or interests concerning the system or its properties meeting their needs and expectations.
- Stakeholders provide opportunities and limitations for the system and are the source of requirements.
- Successful software architecture begins by understanding stakeholders and their goals before making technical decisions.

Good Architecture Starts with “Why” and “Who”

- Before implementation:
 - Understand business goals
 - Understand stakeholder concerns
- Then decide:
 - Technologies
 - Structures
 - Design approaches

If architects ignore “why” and “who”

- Wrong features may be developed
- Scalability issues may appear
- Stakeholder conflicts happen
- Expensive redesign becomes necessary



Architectural Decisions

- What is an AD?

An Architectural Decision (AD) documents important decisions with architectural significance.

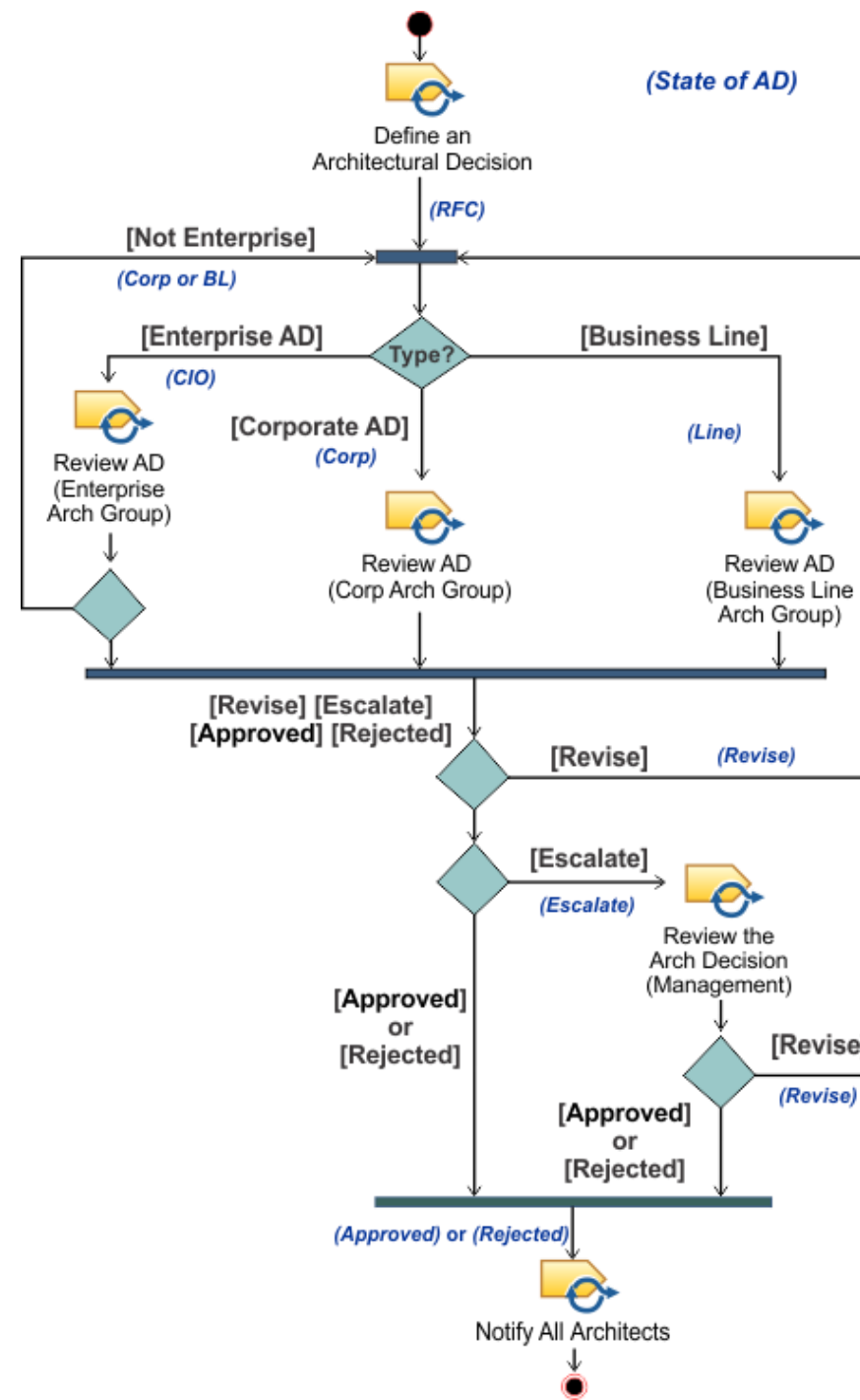
(Decisions that having influence on the selection of hardware, software, the structure of components and integration of components.)

When to do an Architectural Decision?

- Architectural Decisions should be made Early in the software development process, especially during system architecture and design stages.
- When a decision affecting the fundamental structure of a system is taken an Architectural Decision should be created.

However, ADs Are Not One-Time Decisions

AD Process



What problems happen if
important architectural
decisions are not documented?

AD vs ADR ?????

Architectural Decision (Must be documented using the AD Process)	Design Decision (Not documented using the AD Process)
What is the backup / restore architecture used? (SAN vs. Tapes vs. Optical)	The SAN switch design needed in order to support the backup and restore.
Architectural structure decisions such as "Client Server", "n-tier", etc.	Any detailed design decision (e.g. the layout of a user interface, the detailed user steps in a manual task, the detailed format of the data in a component interface).
Software Buy vs. Build (How much customization is needed in the buy option and how easy is it to do so, how agile is the buy option, how agile is the custom option, how complex is it to interface with the buy option)	The detailed format of the data in the interface to the COTS software.
What is the transaction support needed inside a component and how to achieve it?	What are the details to handle this transaction level?
What is the persistence mechanism used (e.g. audit, data purging, data archiving)?	What is the detailed design for that persistence strategy?
What is the Application Framework used? (Struts, home-grown, etc.)?	What are the functional application layers used inside the business logic layer of the application framework?
What is the communication mechanism between components?	What is the detail design of the communications between components?
What is the performance requirement and how is it achieved?	How is this component designed in order to provide the needed performance level?
What authentication mechanism is used by this component?	What is contained in the authentication token?

Activity

- A government organization plans to develop a **Smart City Traffic Management System**.
- The initial architecture team decides to use a centralized cloud-based architecture with AI processing at the main data center.”
- Suppose that you are the architecture review team.
- Your task is to identify:
 1. Missing stakeholders
 2. Stakeholder concerns
 3. Architectural risks
 4. Better architectural decisions

